

«Питомец Финни» — план реализации на Kotlin + Jetpack Compose

Функциональный прототип Android-приложения для формирования финансовых навыков у детей 7–11 лет.

Соответствие ТЗ: локальный профиль без регистрации, офлайн-режим, демонстрационный режим,

подготовка к публикации в RuStore. Серверная часть не используется.

1. Технологический стек и окружение

Компонент	Выбор	Обоснование
Язык	Kotlin 2.x, JVM 17	Официальный стек Android
UI	Jetpack Compose (Material 3)	Декларативное состояние, анимации питомца
Архитектура	MVVM + однонаправленный поток данных (UiState + события)	Тестируемость, простая логика для демо
DI	Hilt	Стандарт, минимум шаблонного кода
БД	Room (SQLite)	Типобезопасная схема профиля и истории периодов
Настройки	DataStore Preferences	Флаги (демо-режим, звук, тестовый профиль)
Сериализация контента	kotlinx.serialization	JSON-контент в assets
Тесты	JUnit 5 / kotest, Turbine (для Flow)	Покрытие игровой экономики
Мин. SDK	26 (Android 8.0)	По ТЗ п. 3.1
Сборка	Gradle Kotlin DSL, assembleRelease	Воспроизводимость, ключ подписи вне репозитория

Разрешения Android: **никакие не запрашиваются** (в ТЗ все функции работают офлайн и без железа).

2. Модульная структура проекта

```
app/
├── core/
│   ├── economy/          # ЧИСТАЯ логика без Android-зависимостей (юнит-тестируемая)
│   │   ├── BudgetEngine.kt    # план vs факт, контроль лимита, остаток
│   │   ├── SavingsEngine.kt   # пополнение, снятие с подтверждением, срок цели
│   │   ├── RewardEngine.kt    # начисления за задания, периодический доход
│   │   └── PetEngine.kt       # состояние (настроение/сытость) и стадии развития
│   ├── content/           # Загрузка и валидация JSON-контента из assets
│   ├── data/              # Room (DAO, entities, мапперы), DataStore, репозитории
│   └── design/            # Тема, цвета, типографика (16 sp+), кнопки ≥48 dp
├── feature/
└── onboarding/           # Онбординг: 3 типа решений (обяз. / желаемое / отложить)
```

pet/	# Создание питомца (9+ комбинаций), главный экран
budget/	# План бюджета на период (3 направления)
shop/	# Каталог покупок, покупка с подтверждением
goals/	# Цели, накопления, срок достижения
tasks/	# Задания-мини-сценарии
history/	# История, итоги периода, справочник терминов
adult/	# Раздел для взрослого (барьер: удержание кнопки)
assets/content/	
tasks.json	# ≥6 заданий по 3 темам
shop.json	# ≥8 товаров двух типов
goals.json	# ≥3 цели
glossary.json	# справочник терминов

Ключевое требование ТЗ (п. 3.4): **контент отделён от логики** — все задания/товары/цели живут в JSON, добавление задания = новая запись в файле без переработки кода.

3. Поэкранный карта (12 шагов сквозного сценария)

#	Экран	Содержание	Шаг ТЗ
1	Onboarding (3 карточки)	Цель игры; три типа решений: «обязательное», «желаемое», «отложить». Повторный доступ из главного меню	1
2	Создание профиля	Гостевой режим: игровое имя ребёнка, без телефона/e-mail	2
3	Создание питомца	Имя + конструктор внешности: 3 части × 3 варианта = 9+ комбинаций	3
4	Главный экран	Питомец, баланс, накопления, цель, настроение/сытость, активное задание — всё одновременно; навигация: бюджет, задания, покупки, накопления, прогресс, взрослый	3–4
5	Начисление дохода	Стартовый бюджет + ежедневный игровой доход; каждое начисление — с источником и суммой	4

6	План бюджета	Слайдеры/ степперы: обязательные / необязательные / накопления; контроль суммы $\leq$ баланс, показ остатка; изменение до подтверждения	5
7	Задание	Игровая ситуация с выбором и последствиями + короткое объяснение (правильно и ошибочно) + награда	6
8	Магазин	Карточки товаров: цена, категория, влияние на питомца; подтверждение покупки; отказ при нехватке с объяснением вариантов	7
9	Цели и накопления	Выбор цели, прогресс-бар, пополнение; снятие — только с двойным подтверждением и показом нового срока	8
10	Итоги периода	План vs факт (по 3 направлениям), объяснение состояния питомца, кнопка «Следующий период»	9–10
11	История / прогресс	Завершённые задания, прогресс цели, итоги последнего периода, справочник терминов	11

12	Раздел взрослого	Барьер: удержание кнопки 3 сек; цели приложения, пройденные темы, прогресс, сброс/удаление профиля; без негативных оценок	12
----	------------------	---	----

UX-обязательства из ТЗ (п. 3.6): шрифт ≥ 16 sp, элементы ≥ 48 dp, цвет не единственный носитель смысла (иконки + текст), звуки/анимации отключаемы, все разрушающие действия — с подтверждением.

4. Структуры данных

4.1. Room-entities (локальный профиль)

@Entity

```
data class ProfileEntity(
    @PrimaryKey val id: String,           // UUID, генерируется локально
    val childName: String,
    val petName: String,
    val petBody: Int, val petColor: Int, val petAccessory: Int,
    val balance: Int,                    // доступная валюта, монеты
    val savings: Int,                    // накопления (отдельно от баланса)
    val goalId: String?,                 // выбранная цель
    val petStage: Int,                   // 0..N стадий развития
    val mood: Int, val satiety: Int,     // 0..100
    val isTestProfile: Boolean,          // тестовый профиль для экспертизы
)
```

@Entity

```
data class PeriodEntity(
    @PrimaryKey val id: String,
    val profileId: String,
    val index: Int,                      // номер игрового периода
    val planMandatory: Int,              // план: обязательные
    val planOptional: Int,               // план: необязательные
    val planSavings: Int,                // план: накопления
    val factMandatory: Int,
    val factOptional: Int,
    val factSavings: Int,
    val status: String,                  // PLANNED / ACTIVE / CLOSED
    val startedAt: Long, val closedAt: Long?,
)
```

@Entity

```
data class TransactionEntity(
    @PrimaryKey val id: String,
    val profileId: String,
    val periodId: String?,
    val type: String,                   // INCOME_TASK / INCOME_DAILY /
    PURCHASE_MANDATORY / PURCHASE_OPTIONAL / SAVINGS_IN / SAVINGS_OUT
)
```

```

        val amount: Int,
        val title: String,                // «За задание», «Еда для Финни»...
        val createdAt: Long,
    )

@Entity
data class TaskProgressEntity(
    @PrimaryKey val taskId: String,
    val profileId: String,
    val completed: Boolean,
    val attempts: Int,
    val lastCorrect: Boolean,
)

```

4.2. Контент: tasks.json (пример записи)

```

{
  "id": "task_budget_1",
  "topic": "BUDGET_PLANNING",           // PLANNING | SAVINGS | PURCHASES
  "title": "День рождения Финни",
  "story": "У Финни 30 монет. Корм стоит 12, шарик – 10.",
  "scenario": [
    {
      "step": 1, "text": "Что купим в первую очередь?", "options": [
        {
          "id": "a", "text": "Корм", "effect": {"mood": 0, "satiety": +10},
          "explanation": "Верно! Обязательные расходы – в первую очередь.",
          "correct": true,
        },
        {
          "id": "b", "text": "Шарик", "effect": {"mood": +5, "satiety": -5},
          "explanation": "Финни грустит без шарика, но голоден. Обязательное важнее желанного.",
          "correct": false
        }
      ]
    }
  ],
  "reward": 5
}

```

4.3. Контент: shop.json и goals.json

```

// shop.json – минимум 8 позиций двух типов
{
  "id": "food_1", "name": "Корм «Вкусняшка»", "category": "MANDATORY",
  "price": 10, "effect": {"satiety": +15, "mood": 0},
  "influenceHint": "Финни будет сыт"
}

// goals.json – минимум 3 цели
{
  "id": "goal_bicycle", "name": "Велосипед для прогулки", "price": 100, "icon": "bike"
}

```

5. Формулы игровой экономики

Все формулы — в core/есопоту/, покрыты юнит-тестами (требование п. 3.4).

5.1. План бюджета

Сумма плана = planMandatory + planOptional + planSavings
 Контроль: Сумма плана ≤ balance, иначе блок + подсказка, чего не хватает
 Остаток = balance – Сумма плана (показывается ребёнку при планировании)

5.2. Покупка

if (price > balance) → отказ: "Не хватает X монет. Варианты: отложить, выбрать дешевле, заработать в задании, перенести необязательную покупку"
else balance -= price; fact<категория> += price; применить эффект к состоянию

5.3. Накопления и срок цели

Пополнение: savings += X; balance -= X (внутри периода идёт в factSavings)

Снятие: только после подтверждения; до подтверждения показать
savings' = savings - X и новый срок

Срок цели = $\text{ceil}((\text{price} - \text{savings}) / \text{среднее пополнение за период})$ в игровых периодах

Среднее пополнение = среднее factSavings за последние 3 закрытых периода (или факт текущего, если данных мало).

5.4. Награды

Доход за задание: reward (из tasks.json), начисление = транзакция INCOME_TASK с источником

Ежедневный доход: base = 20 монет, но в ДЕМО-режиме выдаётся по кнопке «Следующий день»

Бонус от взрослого: 0–10 монет, вручную, с подтверждением взрослого (механика команды)

5.5. Состояние питомца (настроение, сытость)

mood' = clamp(mood + Σэффекты покупок + Σэффекты заданий, 0..100)

satiety' = clamp(satiety + Σэффекты еды, 0..100)

Обязательные покупки не куплены к закрытию периода → mood -15, satiety -20

(обратимо, без «смерти»).

5.6. Стадии развития (≥3 стадии)

score = 0.4·(доля периодов с закрытыми обязательными расходами)
+ 0.3·(доля периодов, где |факт-план| ≤ 20% по каждому направлению)
+ 0.3·(доля периодов с factSavings ≥ planSavings)

stage = 0 (Щенок) → 1 (Подросток) при score ≥ 0.6 → 2 (Звезда) при score ≥ 0.8 за 5 периодов

Стадия никогда не понижается ниже достигнутой («безопасная ошибка» — прогресс не обнуляется).

После каждого изменения — короткое объяснение: «Финни вырос! Ты 4 периода подряд откладывал на цель».

6. Демонстрационный режим (критично для экспертизы)

- Включается в разделе взрослого; флаг — в DataStore.
- Убирает календарные ожидания: ежедневный доход выдаётся по кнопке «+день», задания открыты сразу (п. 2.5.8).
- Кнопка «Сбросить тестовый профиль»: возвращает профиль к исходному состоянию (стартовый баланс, stage 0, чистая история) — перепроход шагов 1–12 заново.
- Тестовый профиль помечается isTestProfile = true, чтобы эксперт видел, что данные не «боевые».
- Игровой период в демо-режиме закрывается по кнопке «Завершить период», а не по таймеру.

7. Тестирование

1. **Юнит-тесты** (`core/economy/``): контроль лимита плана, отказ покупки при нехватке, снятие с накоплений без подтверждения, пересчёт стадии, срок цели. Минимум 25 кейсов.
2. **Интеграционные**: Room-миграции, сохранение после «закрыть приложение → открыть» (шаг 11 ТЗ).
3. **Ручной чек-лист на физическом устройстве** (RAM ≥3 ГБ, Android 8+): запуск ≤5 с, отклик ≤1 с, отсутствие падений на шагах 1–12, офлайн-работа.
4. **Матрица соответствия ТЗ** (п. 5 документации): требование → статус → экран/модуль/тест.

8. План работ (4–6 недель, 1–2 разработчика)

Неделя	Результат
1	Проект, тема, навигация, Room + DataStore, экраны 1–3 (онбординг, профиль, создание питомца)
2	Главный экран (4), доходы (5), план бюджета (6) + движки Budget/Reward
3	Задания (7), магазин (8), цели и накопления (9) + контент JSON (6 заданий, 8 товаров, 3 цели)
4	Итоги периода (10), история (11), раздел взрослого (12), стадии питомца
5	Демонстрационный режим, тестовый профиль, юнит-тесты, полировка UX
6	Релизная сборка APK, тест на физическом устройстве, документация, презентация, видео-демо (до 3 мин)

Промежуточная сдача (п. 7.1) возможна после недели 4: ≥3 ключевых экрана, README, таблица статусов требований.

9. Документация и сдача (по разделу 5 ТЗ)

- README: назначение, состав репозитория, быстрый запуск, сборка релизного APK (`./gradlew assembleRelease`, ключ подписи — локально, не в git).
- DOCX/PDF-документ: архитектура, структура данных, формулы (раздел 5 настоящего плана), матрица соответствия, карта контента, обоснование UX/UI, перечень разрешений (0 шт.), тест-кейсы, ограничения, лицензии ассетов.
- Презентация 8–12 слайдов по п. 4 ТЗ.
- Резервная видеозапись демонстрации до 3 минут.

10. Риски и ограничения

- Контент (сценарии заданий, рисунки питомца) — главный объём работ; смягчение: модульный питомец из 3 частей, генерация/свободные ассеты с фиксацией лицензий.

- Compose-анимации настроения: ограничить 2–3 простыми (масштаб, «прыжок», смена эмодзи-наклейки), чтобы уложиться в бюджет демо.
- Таблицы/графики «план vs факт»: кастомные composable, без тяжёлых chart-библиотек.